T3 Exercises – Fateme Firouznia

1. **Could you please explain the security mechanisms used by package repositories to prevent the installation of corrupted or tampered packages on a system?**

   Package repositories use several security mechanisms to prevent the installation of corrupted or tampered packages. The core mechanisms are digital signatures, checksum verification, and a trust chain based on repository signing keys.

   Each repository provides a signed Release file that contains checksums of the available packages. The package manager first verifies the GPG signature of the Release file to confirm that the metadata comes from a trusted source. After downloading a package, its checksum is compared against the value listed in the Release file.

   If the signature verification fails or the checksum does not match, the package installation is rejected. This ensures both the authenticity of the repository and the integrity of the downloaded packages.

2. **We are using the following APT source list in /etc/apt/sources.list: Could you explain the role of each repository component (e.g., main, universe, multiverse, security, updates, backports)?**

   deb http://ir.archive.ubuntu.com/ubuntu focal main restricted deb http://ir.archive.ubuntu.com/ubuntu focal-updates main restricted deb http://ir.archive.ubuntu.com/ubuntu focal universe deb http://ir.archive.ubuntu.com/ubuntu focal-updates universe deb http://ir.archive.ubuntu.com/ubuntu focal multiverse deb http://ir.archive.ubuntu.com/ubuntu focal-updates multiverse deb http://ir.archive.ubuntu.com/ubuntu focal-backports main restricted universe multiverse deb http://security.ubuntu.com/ubuntu focal-security main restricted deb http://security.ubuntu.com/ubuntu focal-security universe deb http://security.ubuntu.com/ubuntu focal-security multiverse

   In the APT sources list, repositories are divided into components that group packages based on support level and licensing.

   The **main** component contains officially supported Ubuntu packages and is enabled by default.
   The **restricted** component includes proprietary or license-restricted software that is still officially supported.

   The **universe** component provides community-maintained software without official support.

The **multiverse** component contains software with legal or licensing restrictions and no official support.

The **updates** repositories deliver stable bug fixes and non-breaking improvements.
The **security** repositories provide critical security patches.
backports offer newer versions of packages from later Ubuntu releases, mainly for optional use.

3. What is the difference between KVM and QEMU? Also, what exactly is qemu-kvm?

KVM is a Linux kernel module that enables hardware-assisted virtualization using CPU features such as Intel VT-x or AMD-V. It allows virtual machines to run with near-native performance.

QEMU is a software-based emulator and virtualizer that can simulate hardware components. When used alone, it relies on software emulation and has lower performance.

qemu-kvm refers to using QEMU together with KVM, where QEMU handles device emulation and VM management, while KVM provides hardware acceleration through the kernel.

4. What are the differences between a virtual machine and a container? For each of the following scenarios, which one would you recommend and why?

Virtual machines and containers are both used for application isolation, but they differ in architecture.

A virtual machine runs a full operating system on top of a hypervisor, with its own kernel. This provides strong isolation but requires more system resources.
Containers share the host kernel and isolate only the user space, making them lightweight and faster to start.

For a **database under heavy load with frequent disk writes**, a virtual machine is recommended due to better isolation and I/O control.
For a **stateless web application**, containers are more suitable because they scale easily.
For a **monolithic web application with high resource demands**, a virtual machine is preferred.
Applications that require **specific hardware drivers** are better suited to virtual machines.
GUI applications work more reliably in virtual machines.

Applications depending on **multiple system services initialized at boot** are better handled in virtual machines.
Applications requiring **custom kernel modules** must run in virtual machines.
Lightweight applications are ideal candidates for containers.
Applications that need **direct interaction with other processes** benefit from containers.

In general, virtual machines provide stronger isolation, while containers focus on efficiency and scalability.

5. **What is the difference between a hashing algorithm and an encryption algorithm?**

Hashing and encryption algorithms serve different purposes in data security.

Hashing transforms data into a fixed-length value using a one-way process. It is not reversible and is mainly used to verify data integrity. Even a small change in the input results in a completely different hash value.

Encryption converts data into an encoded form that can be reversed using a specific key. Its primary purpose is to protect data confidentiality, allowing the original data to be recovered when needed.

In practice, hashing is commonly used for integrity checks and password storage, while encryption is used to secure sensitive data during storage or transmission.

6. **I have a directory with many configuration files. How can I generate checksums to detect even a single-character change in any file?**

To detect even a single-character change in configuration files, checksum algorithms such as SHA-256 can be used. A checksum is a fixed-length value that uniquely represents the content of a file.

By generating checksums for all configuration files and saving them in a reference file, it becomes possible to verify file integrity at any later time. Tools like sha256sum can be used to calculate checksums for every file in a directory.

For example, checksums can be generated using the following command:

```
find /path/to/configs -type f -exec sha256sum {} \; > checksums.txt
```

Later, the integrity of the files can be verified with:

```
sha256sum -c checksums.txt
```

If even a single character in any file is modified, the calculated checksum will differ from the stored value, and the verification process will report the file as changed.